

DriveGuard — Student Edition

DriveGuard is a Computer Vision project focused on driver monitoring. It leverages **YOLOv8** and **MediaPipe** to detect risky driving behaviors such as drowsiness, distraction, and phone use in real-time.

Designed as a modern full-stack application, the project features a Python/FastAPI backend and a Vue.js frontend, orchestrated via Docker.

1. Features

- **Drowsiness Detection:** Uses Eye Aspect Ratio (EAR) to detect prolonged eye closure.
- **Yawn Detection:** Uses Mouth Aspect Ratio (MAR) to detect fatigue.
- **Distraction Detection:** Tracks head pitch and yaw to ensure the driver's eyes are on the road.
- **Phone Usage:** Utilizes a YOLOv8 object detection model to identify cell phone usage.
- **Risk Scoring:** Evaluates the session dynamically and assigns a normalized risk score (0-100).
- **Web Dashboard:** View live sessions, session history, and detailed analytics.

2. Architecture Overview

- **Computer Vision Engine:** Python-based pipeline utilizing OpenCV, MediaPipe (Face Mesh), and YOLOv8 (Object Detection).
- **Backend API:** FastAPI application connected to an SQLite database via SQLAlchemy, managing jobs and persisting session results.
- **Frontend:** Vue.js Single Page Application (SPA) providing an interactive operations dashboard.
- **Deployment:** Configured via Docker Compose for easy evaluation and scaling.

3. Getting Started

Prerequisites

- Python 3.10+
- Node.js 18+ (for frontend development)
- Docker & Docker Compose (optional, but recommended for full-stack deployment)
- Standard webcam (for live monitoring)

Installation (Local Development)

1. Clone the repository:

```
git clone https://github.com/anshere0/CV-24BAI10709-DriveGuard.git
cd DriveGuard
```

2. Setup Python Environment:

```
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

3. **Database Setup:** DriveGuard defaults to SQLite for easy local setup, but can be configured for PostgreSQL via `config.yaml`.

4. **Start the Backend:**

```
uvicorn backend.app.main:app --reload --port 8000
```

5. **Start the Frontend:**

```
cd frontend
npm install
npm run dev
```

Navigate to `http://localhost:5173` to access the dashboard.

Testing

To run the test suite for the computer vision pipeline and backend API:

```
# Ensure your virtual environment is activated
pytest
```

4. Running via CLI

DriveGuard provides a headless CLI for rapid video analysis without starting the web dashboard.

```
# Analyze a pre-recorded video
python main.py run samples/demo.mp4

# Run live analysis via webcam (Camera 0)
python main.py run 0
```

5. Docker Deployment

To launch the entire stack (FastAPI Backend, Vue Frontend, SQLite Database) using Docker:

```
docker compose up --build
```

Access the application at <http://localhost:5173>.

6. Project Structure

- [cv_engine/](#): Core algorithms (YOLO, MediaPipe, Tracking, Risk Scoring).
- [backend/](#): FastAPI server, database models, and route definitions.
- [frontend/](#): Vue 3 application.
- [models/](#): YOLOv8 and MediaPipe model files.
- [docs/](#): System architecture, UML diagrams, and academic report materials.
- [tests/](#): Pytest suite covering the CV pipeline and API.

7. Configuration

Behavior thresholds, scoring weights, and infrastructure endpoints are entirely managed via [config.yaml](#). Refer to [.env.example](#) for overriding sensitive variables.

8. License

This project is licensed under the MIT License.